Git & Branching Workflow

Contents

Git & Branching Workflow **1**
- Scope . . . 1
- Official requirements . . . 1
- Project policy . . . 1
- Checklist . . . 2
- References . . . 2
- Related . . . 2

Git & Branching Workflow

Scope

Branching, commits, review and repository hygiene for a small co-located team.

Official requirements

- Work is submitted in the git repository; only repository contents count at defence.
- Folder and file names must be exactly as expected — verify before submission.
- Credentials must never enter history.

Project policy

Branches

- `main` is always in a demonstrable state. No direct pushes.
- Feature branches: `feat/<context>-<short-slug>`, e.g. `feat/vote-cast-endpoint`.
- Fix branches: `fix/<short-slug>`. Chore/docs: `chore/…`, `docs/…`.
- Rebase onto `main` before opening a pull request. Merge with a merge commit so the feature boundary stays visible.

Commits

Conventional Commits: `type(scope): subject`, imperative mood, lower-case subject, no trailing period.

Allowed types: `feat`, `fix`, `refactor`, `test`, `docs`, `chore`, `perf`, `build`, `ci`.

One logical change per commit. A commit that does not build is not a commit.

Pull requests

- Minimum one reviewer. In a two-person team, the other person.
- The description states **what changed and why**, links the ADR or Brain note, and lists manual verification steps.
- CI must be green before merge.

- Reviewer checks against the standard that applies, not personal taste.

Repository hygiene

- Never commit `.env`, `.DS_Store`, build output, or the `reference/` tree.
- A secret that reaches any branch is treated as leaked: rotate the credential first, then rewrite history. Deleting the file is not sufficient.
- `reference/nArchitecture` has its own `.git`. Do not add it as a submodule and do not commit its working tree.

Checklist

- ☐ Branch name matches the convention
- ☐ Commits follow Conventional Commits and each one builds
- ☐ No secrets, no generated files in the diff
- ☐ PR links its Brain/ADR context and names its verification steps

References

- Conventional Commits 1.0.0 — https://www.conventionalcommits.org/en/v1.0.0/
- Pro Git — https://git-scm.com/book/en/v2

Related

`../engineering-baseline/STANDARD.md` · `../../40-security/secrets-management/STANDARD.md`